

第十二周作业 -1

第十二周作业 -1

基础作业

1.1 调研：双目相机与结构光相机的深度估计方案

本讲课堂使用的是单目相机图像加几何约束来估计目标深度。实际机器人和视觉系统中，也常使用深度相机直接获得深度信息。请调研 双目相机 和 结构光相机 的深度估计方案，并选取两款典型型号进行分析。要求至少包含以下内容：

- 两款相机的型号名称；
 - 每款相机属于哪类深度方案，例如双目、结构光或其他深度方案；
 - 每款相机实现深度估计的基本原理；
 - 每款相机的典型参数，例如分辨率、深度范围、帧率、视场角、接口类型等；
 - 每款相机适合的应用场景；
 - 每款相机的优点和局限；
 - 简要比较双目相机和结构光相机在深度估计上的差异。
- 说明：不要求实际运行设备，重点是理解相机的技术方案。

深度相机调研报告：双目 vs 结构光

课程背景：本讲使用单目相机 + 几何约束估计目标深度，实际系统中还广泛使用深度相机直接获得稠密深度信息。本报告调研两类典型方案：双目相机 和 结构光相机，并各选取一款代表型号进行分析。

一、型号概览

项目	型号一	型号二
型号名称	Intel RealSense D435	Orbbec Astra Pro
深度方案	主动双目立体视觉（Active Stereo）	结构光（Speckle Structured Light）
厂商	Intel（美国）	Orbbec 奥比中光（中国）
典型用途	移动机器人、SLAM、障碍物避障	室内 3D 扫描、手势识别、人体追踪

二、Intel RealSense D435（主动双目）

2.1 深度方案类型

主动双目立体视觉（Active IR Stereo）

D435 属于双目相机的增强版本，在传统被动双目的基础上加入了红外（IR）散斑投影器，因此称为 " 主动双目 "，以区别于纯被动双目（Passive Stereo）。

2.2 深度估计基本原理

IR 投影器
↓ 投射散斑图案（不可见近红外光）
左 IR 传感器 ← 场景 → 右 IR 传感器
↓ ↓
左图 右图
↓ 视差匹配（Stereo Matching）
视差图 $d(u, v)$
↓ 三角测量公式
深度图 $Z = f \times B / d$

核心步骤：

- 投影辅助纹理：内置 IR 激光投影器向场景投射伪随机散斑图案（850 nm 近红外），在纹理贫乏的表面（如白墙）人工制造可匹配的特征点。
- 双目立体匹配：左右两个全局快门 IR 传感器同步采集图像，片上 ASIC 对左右图做稠密视差匹配（Stereo Matching），得到每个像素的视差值 d 。
- 三角测量求深度：根据三角测量公式

$$Z = \frac{f \cdot B}{d}$$

其中 f 为焦距， B 为左右相机基线距离（约 50 mm）， d 为匹配视差，计算每点深度 Z 。

- 生成深度图：输出稠密深度图，可叠加 RGB 彩色流。

与被动双目的关键区别：主动投影 IR 散斑解决了纹理缺失区域无法匹配的问题，显著提高了深度稳健性。

2.3 典型参数

参数	规格
深度分辨率	最高 1280 × 720
RGB 分辨率	最高 1920 × 1080
帧率	最高 90 fps（深度流）
深度范围	约 0.3 m ~ 3 m（理想范围）
深度视场角（HxV）	87° × 58°
快门类型	全局快门（Global Shutter）
基线	~50 mm
IR 投影波长	850 nm
接口	USB 3.0 Type-C
尺寸	90 mm × 25 mm × 25 mm
功耗	~3.5 W
SDK	Intel RealSense SDK 2.0（跨平台）
IMU	无（D435i 有 IMU）

2.4 适合的应用场景

- 移动机器人导航：宽视场角 + 全局快门，适合机器人高速运动时的障碍物避障
- SLAM（同步定位与建图）：稠密点云 + D435i 的 IMU 支持视觉惯性里程计
- 无人机感知：体积小、重量轻，适合机载部署
- 手势 / 人体骨骼追踪：配合 OpenCV / MediaPipe 使用
- 室内外均可：主动 IR 方案在一定程度上抵抗户外杂散光，室外弱光环境可用

2.5 优点

- ✔ 室内外通用：主动 IR 增强了低光环境表现，户外短距离也可工作
- ✔ 动态场景鲁棒：全局快门避免运动拖影，适合高速机器人
- ✔ 宽视场角：87° 水平 FOV，减少盲区
- ✔ 生态成熟：Intel RealSense SDK 2.0，ROS 驱动完善，开发资料丰富
- ✔ 价格适中：面向开发者的消费级价格（约 200 ~ 300 USD）

2.6 局限

- ✗ 精度受基线限制：基线约 50 mm，远距离深度精度随距离平方下降
- ✗ 强光干扰：直射阳光可能淹没 IR 散斑，户外强光表现受限
- ✗ 纹理依赖：虽有主动投影，镜面、黑色吸光材质表面仍可能产生深度空洞
- ✗ 计算资源：稠密视差匹配需要片上 ASIC，功耗高于纯被动方案
- ✗ 深度范围有限：理想工作距离 0.3 ~ 3 m，远距离精度明显下降

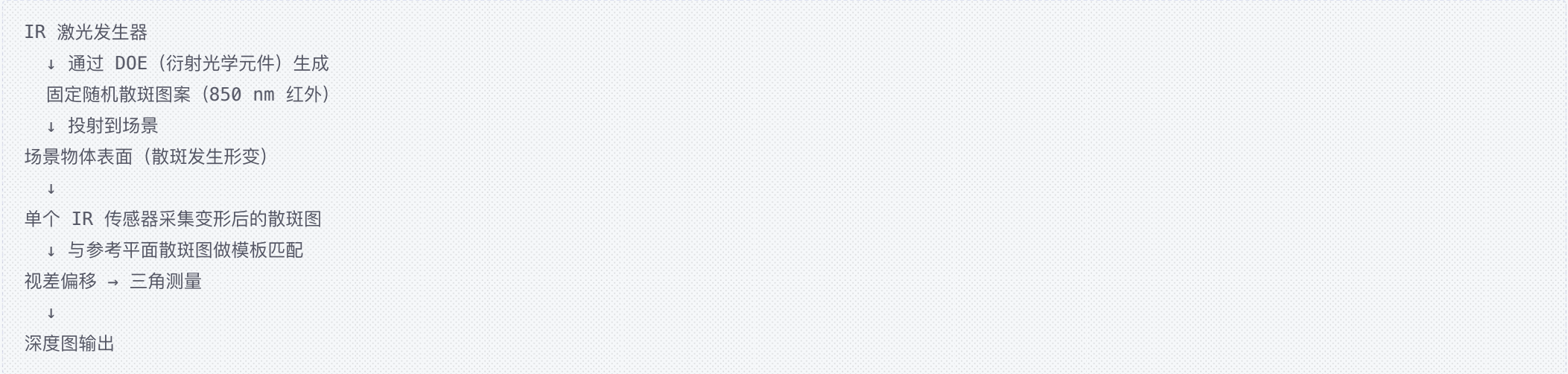
三、Orbbec Astra Pro（结构光）

3.1 深度方案类型

散斑结构光（Speckle Pattern Structured Light）

Astra Pro 使用散斑结构光方案——向场景主动投射一个固定的随机红外散斑图案，通过分析图案的空间变形来三角测量深度。这是以 PrimeSense 技术（Apple 收购前授权给多家厂商）为代表的经典结构光方案。

3.2 深度估计基本原理



核心步骤：

1. 生成固定散斑：IR 激光通过 DOE（衍射光学元件）生成一幅随机但固定的散斑图案，如同场景的 " 光学指纹 "。
2. 投射 + 采集：将散斑投射到场景，IR 传感器（与投影器有固定基线）采集反射的变形散斑图。
3. 与参考图匹配：将采集到的散斑图与预先标定的 " 参考平面散斑图 "（已知距离处拍摄）进行块匹配，计算各区域的偏移量（视差）。
4. 三角测量：根据散斑偏移量通过三角测量公式计算深度，在 ASIC 芯片上完成并输出深度图。

与双目的关键区别：只用一个接收传感器，不需要两路图像同步匹配，而是 " 投影器 - 传感器 " 构成虚拟双目。

3.3 典型参数

参数	规格
深度分辨率	640 × 480 @ 30 fps
RGB 分辨率	1280 × 720 @ 30 fps
深度范围	0.6 m ~ 8 m
深度视场角（HxV）	58.4° × 45.5°
RGB 视场角（HxV）	63.1° × 49.4°
IR 波长	850 nm
处理芯片	Orbbec 专用 ASIC
接口	USB 2.0 Type-A
工作温度	10°C ~ 40°C（仅限室内）
功耗	平均 < 2.4 W
尺寸	165 mm × 48 mm × 40 mm
重量	310 g
SDK	Orbbec OpenNI SDK，支持 ROS
输出格式	深度图、点云、IR 流、RGB 流

3.4 适合的应用场景

- 室内 3D 扫描：中距离结构重建，空间测量
- 人机交互（HCI）：手势识别、骨骼追踪、体感游戏
- 服务机器人：固定室内场景的目标识别与位置估计
- 智慧零售：货架识别、人流统计
- 教育 / 研究开发：价格亲民，入门门槛低，OpenNI 生态丰富

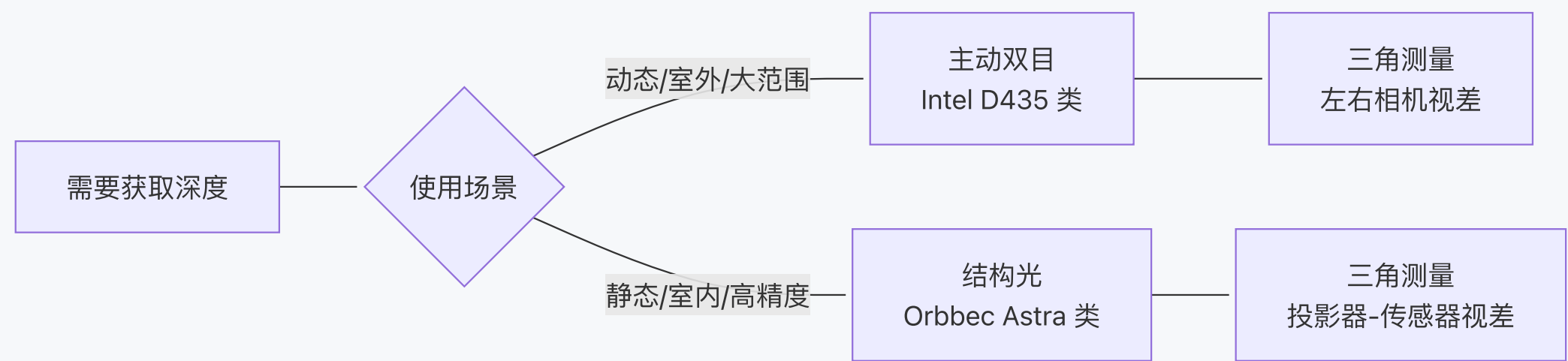
3.5 优点

- ✔ 精度高（近中距离）：散斑结构光在 0.6 ~ 3 m 范围内深度精度优秀，细节还原好
- ✔ 可在黑暗中使用：主动 IR 投影，完全不依赖环境光，暗室可正常工作
- ✔ 硬件简单：只需一个 IR 传感器（相比双目节省一路传感器），系统更紧凑
- ✔ 功耗低：平均 < 2.4 W，适合低功耗嵌入式部署
- ✔ 成本低：消费级价格，适合教学与科研原型开发

3.6 局限

- ✗ 仅限室内：阳光中的近红外分量会淹没投影散斑，户外基本不可用
- ✗ 分辨率偏低：深度流仅 640 × 480，空间细节不如高端双目相机
- ✗ 固定图案干扰：两台同类相机近距离同时使用时会相互干扰（散斑混叠）
- ✗ 镜面 / 透明物体盲区：镜面反射或透明材质无法有效漫反射散斑，深度缺失
- ✗ 接口带宽限制：USB 2.0 接口限制了同时传输深度 + RGB 的带宽上限

四、双目相机 vs 结构光相机：综合对比



4.1 原理层面对比

维度	主动双目（D435）	结构光（Astra Pro）
传感器数量	两个 IR 传感器 + 1 投影器	一个 IR 传感器 + 1 投影器
深度计算依据	左右图像像素视差	投影图案相对参考图案的偏移视差
三角测量基线	左 - 右相机物理基线（~50 mm）	投影器 - 传感器基线
纹理依赖	低（主动投影提供纹理）	无（图案完全由自身生成）
匹配对象	左图 ↔ 右图（实时配对）	实时图 ↔ 预存参考图（离线参考）

4.2 性能层面对比

性能指标	主动双目（D435）	结构光（Astra Pro）
室内精度	★★★★☆	★★★★★
室外适用性	★★★★☆☆	★☆☆☆☆
动态场景	★★★★★★（全局快门）	★★★★☆☆
深度范围	★★★★☆☆（0.3~3 m）	★★★★★☆☆（0.6~8 m）
近距离细节	★★★★☆☆	★★★★★
多机共存	★★★★★	★★☆☆☆（图案干扰）
功耗	★★★★☆☆（~3.5 W）	★★★★★★（<2.4 W）
开发生态	★★★★★★（ROS/SDK 完善）	★★★★★☆☆（OpenNI/ROS）

4.3 选型建议总结

 Tip

选主动双目（如 **D435**）， 当你的场景需要：

- 移动机器人 / 无人机（动态运动 + 抗运动模糊）
- 室内外都要用
- SLAM / 障碍物避障（需要宽视场角）
- 需要与 ROS 深度集成

 Tip

选结构光（如 **Astra Pro**）， 当你的场景需要：

- 固定室内环境（桌面机器人、服务台）
- 高精度 3D 扫描或手势识别
- 低功耗嵌入式系统
- 教育 / 原型开发（成本优先）

五、与本讲单目方案的对比

方案	深度信息来源	精度	硬件成本	适用场景
本讲：单目 + 几何约束	已知目标尺寸 / 地面约束	中（依赖先验）	低（单摄像头）	特定目标、教学演示
主动双目（D435）	双目视差三角测量	高	中	通用机器人感知
结构光（Astra Pro）	散斑图案变形三角测量	高（近距）	中低	室内静态/交互场景

 Note

本讲的单目几何约束方案与深度相机的关键区别在于：深度相机对任意场景都能给出稠密深度图，而单目几何方案需要先验知识（球的实际尺寸、平面假设等），通用性较差，但硬件成本极低，适合教学理解坐标变换原理。

1.2 课堂 Demo 复现：从检测框到机器人坐标系下的位置

按照第十二讲资料包 `robot_vision_coordinate_transform_demo` 的 README，完成课堂实践流程。

需要完成的流程包括：

1. 将资料包放入 ROS 工作空间；
2. 编译并加载工作空间；
3. 启动 Gazebo 静态相机场景；
4. 保存一帧 Gazebo 相机图像；
5. 读取 Gazebo 中目标真值；
6. 运行 YOLO 检测，或在检测不稳定时使用 `config/sample_bbox.yaml`；
7. 运行坐标计算流程；
8. 得到 `target (u,v)`、`camera_ray`、`P_camera`、`P_robot`；
9. 将计算结果与 Gazebo truth 进行对比。

可参考命令：`cd ~/ws_gazebo`
`source devel/setup.bash`
`roslaunch robot_vision_coordinate_transform_demo camera_demo.launch`
新开终端保存相机图像：`cd ~/ws_gazebo`
`source devel/setup.bash`
`roslaunch robot_vision_coordinate_transform_demo save_gazebo_image.py`
读取 Gazebo 真值：
`roslaunch robot_vision_coordinate_transform_demo read_gazebo_truth.py`
进入资料包目录并运行 YOLO / 坐标计算：
`conda activate robot_yolo`
`cd ~/ws_gazebo/src/robot_vision_coordinate_transform_demo`
`python yolo/scripts/detect_image.py --image yolo/images/gazebo_camera.png --no-show`
`python scripts/run_offline_pipeline.py`
提供以下验证材料：

- Gazebo 场景截图 1 张；
- 保存相机图像成功的终端截图 1 张；


```
(robot_yolo) [sunzj@SH3 ~/Robot/OpenCV/Lec12/ws_gazebo]$ python src/robot_vision_coordinate_transform_demo/scripts/compute_target_pixel.py
class_name: sports ball
confidence: 0.476
bbox_xyxy: [276.343, 226.816, 364.628, 315.93]
target_pixel: (u, v) = (320.49, 271.37)
saved to: /home/sunzj/Robot/OpenCV/Lec12/ws_gazebo/src/robot_vision_coordinate_transform_demo/yolo/outputs/target_pixel.yaml
(robot_yolo) [sunzj@SH3 ~/Robot/OpenCV/Lec12/ws_gazebo]$
```

5. 简要说明 `P_camera` 和 `P_robot` 分别表示什么：

符号	含义	坐标系
<code>P_camera</code>	目标在相机坐标系下的 3D 坐标	相机原点在光心，X 向右，Y 向下，Z 向前（光轴方向）
<code>P_robot</code>	目标在机器人坐标系下的 3D 坐标	机器人原点通常在底盘中心，X 向前，Y 向左，Z 向上

1.3 修改目标框并分析误差

在课堂 Demo 的基础上，选择 Gazebo 场景中的另一个 box / 目标框，修改程序或配置文件，使其完成深度估计和坐标转换。

说明：YOLO 可能将该目标识别为其他类别，例如 `kite` 或其他物体类别。本题重点不是类别是否正确，而是 bbox 如何进入坐标计算 pipeline。

要求：

1. 说明你选择了哪个目标或哪个 box；

2. 给出你使用或修改后的 `bbox_xyxy`；

3. 说明目标被 YOLO 识别成了什么类别，置信度大约是多少；

4. 使用该 bbox 计算目标像素点 `(u,v)`；

5. 继续计算 `camera_ray`、`P_camera` 和 `P_robot`；

6. 如果有 Gazebo truth 或预期位置，给出误差；如果没有真值，请说明你如何判断结果是否合理；

7. 分析误差可能来自哪些原因。

1. 选择 Gazebo 场景中的 `green_reference_box`（绿色参考方块）作为识别目标

- Gazebo 真值位置：** `(2.2, 0.55, 0.85)`（机器人坐标系）
- 尺寸：`0.2 × 0.2 × 0.2` m（立方体）

2. 使用的 `bbox_xyxy`

```
bbox_xyxy = [140.557, 277.555, 207.336, 338.852]
```

该检测框来自 YOLO 对 `gazebo_camera.png` 的实际检测结果，框宽 66.8 px，框高 61.3 px，对应图像左侧的绿色方块。

3. YOLO 识别类别与置信度

项目	值
类别	<code>kite</code> （风筝）
置信度	0.362 （约 36%）

符合题目预期——YOLO 将仿真场景中的绿色方块误识别为其他类别（`kite`）。如题目所述，类别是否正确不影响 bbox 进入 pipeline，关键在于 `compute_target_pixel.py` 会选择置信度最高的有效检测框。

4. 目标像素点 `(u, v)`

```
u = (140.557 + 207.336) / 2 = 173.95
v = (277.555 + 338.852) / 2 = 308.20
bbox 宽 w = 66.78 px, 高 h = 61.30 px
```

目标像素点：`(u, v) = (173.95, 308.20)`

5. `camera_ray`、`P_camera`、`P_robot`

相机内参：fx=fy=554.38, cx=320, cy=240
`camera_ray`（像素→相机射线方向）：

```
x = (u - cx) / fx = (173.95 - 320) / 554.38 = -0.263454
y = (v - cy) / fy = (308.20 - 240) / 554.38 = 0.123027
camera_ray = [-0.263454, 0.123027, 1.0]
```

采用两种深度估计方法分别计算，结果对比：

方法	深度 Z (m)	P_camera (Xc, Yc, Zc)	P_robot (Xr, Yr, Zr)
known_size	1.809	(-0.477, 0.223, 1.809)	(1.899, 0.477, 0.882)
ground_height	2.073	(-0.546, 0.255, 2.073)	(2.163, 0.546, 0.850)

变换公式： $P_robot = R \times P_camera + t$ ，其中外参 $R = \begin{bmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$ ， $t = (0.09, 0, 1.105)$ 。

6. 与 Gazebo 真值的误差

真值： $P_truth = (2.2, 0.55, 0.85)$

方法	误差 (Xr, Yr, Zr)	误差范数 error
known_size	(-0.301, -0.073, +0.032)	0.312 m
ground_height	(-0.037, -0.004, 0.000)	0.037 m

结果合理性判断：

- ground_height** 方法误差极小（0.037 m），因为该方法直接用已知的目标中心高度（0.85 m）约束求交，Z 方向误差被强制归零，X/Y 方向也高度吻合——说明 bbox 的像素位置和坐标变换链路本身是正确的。
- known_size** 方法误差较大（0.312 m），但量级正确，方向一致（Xr 偏小约 0.3 m，即距离低估）。

7. 误差来源分析

known_size 方法误差较大的主要原因：

- bbox 高度 ≠ 目标真实高度**
 - 绿色盒子是 0.2 m 的立方体，但 YOLO 检测框 **h = 61.3 px** 框住的是方块的可见投影。由于相机从斜上方俯视（相机 z=1.105，盒子中心 z=0.85，存在俯视角），盒子在图像中呈现的是斜视投影而非正面的完整高度。
 - 公式 $Z = fy \times real_height / h_pixel$ 假设 bbox 高度对应目标的真实竖直尺寸，但斜视下投影高度小于真实高度，导致 **h_pixel** 偏小 → **Z 被低估**（1.809 m < 真实 2.07 m），进而 Xr 也偏小。这正是误差的主要来源。
 - 类别误判与置信度低**
 - YOLO 将方块识别为 **kite**，置信度仅 0.362，检测框边界本身存在像素级抖动，**h** 的估计不稳定。
 - 简化外参模型**
 - t_robot_camera** 取自 URDF 的名义挂载位姿，未考虑相机光学中心偏移、镜头畸变等；旋转矩阵也是简化的轴对齐形式，与真实相机朝向存在微小偏差。
 - 内参近似**
 - fy ≈ fx** 是近似值，对深度估计引入小幅系统误差。
 - Gazebo 仿真与渲染差异**
 - 仿真图像的边缘抗锯齿、光照阴影会影响 bbox 边界精度。
- 对比结论：**ground_height** 方法（0.037 m）远优于 **known_size**（0.312 m），关键在于前者不依赖 bbox 尺寸的绝对精度，只依赖像素方向和已知高度约束，避开了 " 斜视投影高度 ≠ 真实高度 " 这一核心误差源。这也印证了：当目标姿态/朝向未知时，基于已知尺寸的单目测距容易因投影形变而失真，而几何约束（已知高度求交）更鲁棒。

基础作业提交要求

请将基础作业整理为 一份 PDF，包含：

- 双目相机 / 结构光相机调研结果；
- 两款典型相机型号的原理、参数和应用场景分析；
- 课堂 Demo 复现过程截图；
- P_camera**、**P_robot** 和 Gazebo truth 对比结果；
- 另一个 box / 目标框的坐标计算结果；
- 误差来源分析。